

GIÁO TRÌNH TỰ HỌC

BÀI 1

MÔ HÌNH YOLO26-SEG, VMAMBA VÀ CÁC CHỈ SỐ ĐÁNH GIÁ

*Kiến thức nền và liên hệ với đề tài
phân đoạn polyp đại trực tràng trên bộ dữ liệu Kvasir BG20*

Tháng 10 năm 2026

MỤC LỤC

LỜI MỞ ĐẦU	4
Học theo nguyên lý Pareto	4
CHƯƠNG 1. KIẾN THỨC NỀN VỀ HỌC SÂU	6
1.1. Máy tính nhìn thấy một bức ảnh như thế nào	6
1.2. Học có giám sát và mạng nơ-ron	6
1.3. Mạng nơ-ron tích chập	8
1.4. Những linh kiện quen thuộc trong mạng hiện đại	9
1.5. Học chuyển giao	9
1.6. Cơ chế attention và vấn đề chi phí	10
CHƯƠNG 2. BÀI TOÁN PHÂN ĐOẠN THỰC THỂ	11
2.1. Ba mức độ hiểu một bức ảnh	11
2.2. Tư tưởng của YOLO	11
2.3. Bài toán cụ thể của đề tài	12
CHƯƠNG 3. KIẾN TRÚC YOLO26-SEG	13
3.1. Bức tranh tổng thể	13
3.2. Backbone và đặc trưng đa tỉ lệ	13
3.3. Neck: trộn đặc trưng nhiều tỉ lệ	14
3.4. Head: dự đoán khung và lớp	14
3.5. Mặt nạ được tạo từ prototype và hệ số	14
3.6. Học không cần NMS với hai nhánh dự đoán	16
3.7. Các thành phần của hàm mất mát	17
3.8. Cấu hình huấn luyện của đề tài	17
CHƯƠNG 4. TỪ MÔ HÌNH KHÔNG GIAN TRẠNG THÁI ĐẾN VMAMBA	18
4.1. Ý tưởng của mô hình không gian trạng thái	18
4.2. Rời rạc hóa và bước thời gian	18
4.3. Tính chọn lọc: điểm mới của Mamba	19
4.4. Tính toán song song	19
4.5. VMamba và phép quét bốn hướng	19

CHƯƠNG 5. KHỐI TSVM TRONG ĐỀ TÀI	21
5.1. Động cơ và vị trí đặt khối	21
5.2. Nhánh toàn cục	21
5.3. Nhánh hình dạng và biên có hướng	22
5.4. Cổng điều biến	22
5.5. Hợp nhất, tinh chỉnh và kết nối tắt	22
5.6. Vỏ bọc C2TSVMamba	23
5.7. Các chế độ thí nghiệm loại bỏ	23
5.8. Nhìn nhận một cách phản biện	23
CHƯƠNG 6. CÁC CHỈ SỐ ĐÁNH GIÁ	25
6.1. IoU: thước đo độ trùng khớp	25
6.2. Khi nào một dự đoán được coi là đúng	25
6.3. Precision và Recall	25
6.4. Đường Precision–Recall và AP	26
6.5. mAP@50 và mAP@50-95	27
6.6. Hệ số Dice và các bài báo phân đoạn polyp	28
6.7. Đánh giá trên nhiều seed	28
6.8. Chọn epoch tốt nhất	29
CHƯƠNG 7. ĐỌC KẾT QUẢ THỰC NGHIỆM CỦA ĐỀ TÀI	30
7.1. Kết quả 10 seed	30
7.2. Diễn giải đúng mức	31
7.3. Về ma trận nhầm lẫn	31
7.4. Sự cố checkpoint của seed 0, 5 và 8	31
CHƯƠNG 8. TỔNG KẾT VÀ CÂU HỎI ÔN TẬP	33
8.1. Tóm tắt theo nguyên lý Pareto	33
8.2. Những hiểu lầm cần tránh	33
8.3. Câu hỏi ôn tập kèm gợi ý trả lời	33

LỜI MỞ ĐẦU

Giáo trình này dành cho người mới bước vào đề tài phân đoạn polyp bằng YOLO26-seg kết hợp VMamba. Người đọc không cần biết trước về học sâu. Mỗi khái niệm được giới thiệu theo cùng một trình tự: trước hết là một hình ảnh trực giác quen thuộc, sau đó là định nghĩa và công thức ở mức tối thiểu, cuối cùng là cách khái niệm ấy xuất hiện trong mã nguồn và số liệu của đề tài. Mục tiêu là sau khi đọc xong, người học có thể tự giải thích mô hình hoạt động ra sao, đọc được bảng kết quả và trả lời được những câu hỏi phản biện thường gặp.

Những gì giáo trình khẳng định về đề tài đều đã được đối chiếu với mã nguồn, tệp cấu hình và số liệu gốc trong kho mã của nhóm. Ở chỗ nào thông tin chỉ đến từ công bố bên ngoài hoặc chưa kiểm chứng được, giáo trình sẽ nói rõ để người học không dùng nhầm khi viết khóa luận.

Học theo nguyên lý Pareto

Nguyên lý Pareto, còn gọi là quy tắc 80/20, cho rằng khoảng 20% nguyên nhân tạo ra khoảng 80% kết quả. Áp dụng vào việc học, điều đó có nghĩa là có một số ít ý tưởng cốt lõi mà khi nắm được, người học sẽ hiểu phần lớn bức tranh; các chi tiết còn lại chỉ là phần giải thích thêm cho những ý ấy. Với đề tài này, có năm ý cốt lõi như sau.

Ý thứ nhất: mọi mô hình YOLO đều gồm ba phần nối tiếp nhau là backbone, neck và head. Backbone trích xuất đặc trưng từ ảnh, neck trộn đặc trưng ở nhiều tỉ lệ khác nhau, còn head đưa ra kết quả cuối cùng. Mọi cải tiến, kể cả khối TSVM của đề tài, chỉ là thay thế một khối trong ba phần này.

Ý thứ hai: mặt nạ phân đoạn được tạo bằng cách pha trộn. Mô hình học 32 "mặt nạ mẫu" (prototype) dùng chung cho cả ảnh, và mỗi đối tượng chỉ cần 32 con số để pha trộn các mẫu đó thành mặt nạ của riêng mình. Đây là bí quyết giúp YOLO vừa phân đoạn được vừa giữ tốc độ cao.

Ý thứ ba: YOLO26 không cần bước hậu xử lý NMS nhờ có hai nhánh dự đoán, một nhánh "một-nhiều" để học cho dễ và một nhánh "một-một" để dự đoán thật.

Ý thứ tư: VMamba là cách để mỗi điểm ảnh "nhìn" được toàn bộ ảnh bằng cách quét ảnh theo bốn hướng, với chi phí tính toán tăng tuyến tính theo kích thước ảnh. Đề tài đặt khối TSVM, một biến thể của VMamba có thêm nhánh nhận biết hình dạng, vào tầng thứ 10 của mô hình, thay cho khối attention có sẵn.

Ý thứ năm: chỉ số quan trọng nhất là Mask mAP@50-95, được lấy trung bình trên 10 lần huấn luyện với 10 seed khác nhau và được kiểm định thống kê. Kết quả

hiện tại cho thấy TSVM cao hơn mô hình gốc một chút và ổn định hơn, nhưng chênh lệch chưa có ý nghĩa thống kê.

Nếu chỉ có ít thời gian, người học nên đọc kỹ năm ý trên cùng các hộp "Ghi nhớ" trong từng chương. Các chương dưới đây lần lượt xây nền kiến thức (Chương 1 và 2), giải thích YOLO26-seg (Chương 3), VMamba (Chương 4), khối TSVM của đề tài (Chương 5), các chỉ số đánh giá (Chương 6) và cách đọc kết quả thực nghiệm (Chương 7).

CHƯƠNG 1. KIẾN THỨC NỀN VỀ HỌC SÂU

1.1. Máy tính nhìn thấy một bức ảnh như thế nào

Với con người, một bức ảnh nội soi là hình ảnh niêm mạc ruột màu hồng với những nếp gấp và có thể có một khối polyp nhô lên. Với máy tính, bức ảnh đó chỉ là một bảng số. Ảnh được chia thành lưới các điểm nhỏ gọi là điểm ảnh (pixel). Mỗi pixel của ảnh màu được mô tả bằng ba con số ứng với ba kênh màu đỏ, xanh lá và xanh dương (RGB), mỗi số nằm trong khoảng từ 0 đến 255. Một ảnh cao H pixel, rộng W pixel, có 3 kênh màu sẽ được lưu thành một khối số kích thước $H \times W \times 3$.

Trong học sâu, những khối số nhiều chiều như vậy được gọi chung là tensor. Khi huấn luyện, mô hình không xử lý từng ảnh riêng lẻ mà xử lý một nhóm ảnh cùng lúc, gọi là một batch. Nếu batch gồm B ảnh, tensor đầu vào có bốn chiều $B \times C \times H \times W$, trong đó C là số kênh. Thư viện PyTorch quy ước đặt chiều kênh trước chiều cao và chiều rộng, nên người học sẽ thường gặp ký hiệu BCHW. Trong đề tài, mỗi batch gồm 8 ảnh kích thước 640×640 , tức là tensor đầu vào có kích thước $8 \times 3 \times 640 \times 640$.

1.2. Học có giám sát và mạng nơ-ron

Học có giám sát là cách dạy máy bằng ví dụ có đáp án, giống như học sinh luyện đề có lời giải. Ta đưa cho mô hình rất nhiều cặp "ảnh và nhãn đúng", mô hình đưa ra dự đoán, ta đo xem dự đoán sai bao nhiêu rồi điều chỉnh mô hình để lần sau sai ít hơn. Lặp lại quá trình này đủ nhiều lần, mô hình sẽ dần học được quy luật ẩn trong dữ liệu.

Viên gạch nhỏ nhất của mạng nơ-ron là một nơ-ron nhân tạo. Nơ-ron nhận vào một dãy số x , nhân mỗi số với một trọng số w tương ứng, cộng tất cả lại cùng với một độ lệch b , rồi đưa kết quả qua một hàm kích hoạt f :

$$y = f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$$

Các trọng số w và độ lệch b chính là "kiến thức" của mô hình; học nghĩa là tìm ra giá trị tốt cho chúng. Hàm kích hoạt có vai trò bẻ cong mối quan hệ giữa đầu vào và đầu ra. Nếu không có nó, dù xếp chồng bao nhiêu lớp, cả mạng vẫn chỉ tương đương một phép biến đổi tuyến tính đơn giản. Đề tài dùng hàm SiLU, có dạng $f(x) = x \cdot \text{sigmoid}(x)$, một hàm mềm mại giữ lại giá trị dương và ép nhẹ giá trị âm về gần 0.

Nhiều nơ-ron xếp cạnh nhau tạo thành một lớp, nhiều lớp xếp nối tiếp nhau tạo thành một mạng. Mạng được gọi là "sâu" khi có nhiều lớp. Các lớp đầu học những đặc điểm đơn giản như cạnh và màu, các lớp sau ghép chúng thành những đặc điểm phức

tập hơn như kết cấu bề mặt, đường cong, rồi đến hình dạng của cả một khối polyp.

Để biết mô hình sai bao nhiêu, ta dùng hàm mất mát (loss function). Hàm mất mát nhận dự đoán và nhãn đúng rồi trả về một con số: số càng nhỏ thì dự đoán càng sát đáp án. Việc huấn luyện thực chất là đi tìm bộ trọng số làm cho hàm mất mát nhỏ nhất.

Phương pháp tìm kiếm phổ biến là hạ dốc theo gradient (gradient descent). Hãy tưởng tượng ta đang đứng trên sườn núi trong sương mù và muốn xuống thung lũng. Ta không thấy đường, nhưng cảm nhận được mặt đất dốc về phía nào, nên cứ bước một bước nhỏ theo hướng dốc xuống. Gradient chính là "độ dốc" đó: nó cho biết nếu tăng một trọng số lên một chút thì hàm mất mát tăng hay giảm và tăng giảm nhanh đến đâu. Mỗi bước, trọng số được cập nhật theo công thức:

$$w_{\text{mới}} = w_{\text{cũ}} - \eta \cdot (\text{độ dốc của loss theo } w)$$

Hệ số η gọi là tốc độ học (learning rate). Bước quá dài thì có thể nhảy qua đáy thung lũng, bước quá ngắn thì đi mãi không tới. Thuật toán lan truyền ngược (backpropagation) là cách tính hiệu quả độ dốc cho hàng triệu trọng số cùng lúc, đi ngược từ đầu ra về đầu vào theo quy tắc đạo hàm hàm hợp.

Trong thực tế, người ta dùng các bộ tối ưu cải tiến từ ý tưởng hạ dốc. Đề tài dùng AdamW với tốc độ học ban đầu 0,001. AdamW tự điều chỉnh độ dài bước cho từng trọng số dựa trên lịch sử độ dốc, đồng thời có cơ chế "suy giảm trọng số" giúp trọng số không phình quá lớn.

Có ba thuật ngữ về nhịp huấn luyện cần phân biệt. Một bước (iteration) là một lần cập nhật trọng số sau khi xử lý một batch. Một epoch là một lượt mô hình đã xem hết toàn bộ tập huấn luyện. Đề tài huấn luyện 100 epoch; với 1.040 ảnh và batch 8, mỗi epoch gồm 130 bước.

Dữ liệu được chia thành tập huấn luyện (train) để mô hình học và tập kiểm định (validation) để đo xem mô hình làm tốt đến đâu trên những ảnh nó chưa từng học. Sự chia tách này giúp phát hiện hiện tượng quá khớp (overfitting): mô hình "học thuộc lòng" tập huấn luyện nên loss trên tập train rất thấp, nhưng gặp ảnh mới lại làm kém. Dấu hiệu dễ nhận thấy là loss trên tập train tiếp tục giảm trong khi loss trên tập validation bắt đầu tăng lên.

Ghi nhớ

Huấn luyện là lặp lại ba việc: dự đoán, đo sai số bằng hàm mất mát, rồi điều chỉnh trọng số theo hướng làm giảm sai số. Tập validation dùng để kiểm tra mô hình trên ảnh chưa từng học.

1.3. Mạng nơ-ron tích chập

Nếu nối mọi pixel của ảnh $640 \times 640 \times 3$ với mọi nơ-ron của lớp kế tiếp, số trọng số sẽ lên tới hàng tỉ, vừa tốn kém vừa dễ quá khớp. Mạng nơ-ron tích chập (CNN) giải quyết vấn đề này dựa trên một quan sát đơn giản: một đặc điểm như cạnh hay vết đồ trông giống nhau dù nó xuất hiện ở góc nào của ảnh. Vì vậy ta chỉ cần một bộ lọc nhỏ, rồi trượt bộ lọc đó qua khắp ảnh.

Bộ lọc này gọi là kernel, thường có kích thước 3×3 . Tại mỗi vị trí, ta đặt kernel lên một vùng 3×3 của ảnh, nhân từng cặp số tương ứng rồi cộng lại để được một con số. Trượt kernel qua mọi vị trí, ta thu được một bảng số mới gọi là bản đồ đặc trưng (feature map). Hình 1.1 minh họa phép tính này với một kernel phát hiện cạnh dọc: kernel có cột dương bên trái và cột âm bên phải, nên nó cho giá trị lớn ở những chỗ bên trái sáng hơn bên phải.

a. Ảnh vào 5×5					b. Kernel 3×3			c. Bản đồ đặc trưng 3×3		
7	0	1	2	1	1	0	-1	5	4	8
7	7	5	0	0	1	0	-1	-6	12	13
2	3	5	4	2	1	0	-1	-11	4	12
1	6	6	0	1						
4	3	7	4	3						

Hình 1.1. Minh họa phép tích chập với kernel 3×3 phát hiện cạnh dọc

Điểm mấu chốt là giá trị trong kernel không do con người chọn mà do mạng tự học. Một lớp tích chập thường có hàng trăm kernel, mỗi kernel học phát hiện một kiểu đặc điểm riêng, và mỗi kernel tạo ra một kênh trong bản đồ đặc trưng đầu ra.

Có vài khái niệm đi kèm cần nắm. Bước trượt (stride) là số pixel kernel dịch đi sau mỗi lần tính. Với stride bằng 2, kernel nhảy cách một ô, nên bản đồ đặc trưng đầu ra chỉ còn một nửa chiều cao và một nửa chiều rộng; YOLO dùng cách này để thu nhỏ ảnh dần qua các tầng. Phần đệm (padding) là việc thêm viền số 0 quanh ảnh để kernel có thể đặt được ở mép ảnh. Vùng tiếp nhận (receptive field) là vùng ảnh gốc mà một điểm trong bản đồ đặc trưng "nhìn thấy". Một lớp 3×3 chỉ nhìn vùng 3×3 , nhưng xếp nhiều lớp và thu nhỏ ảnh dần thì vùng nhìn của các lớp sâu sẽ mở rộng ra rất lớn.

Hai biến thể của tích chập xuất hiện nhiều trong khối TSVM. Tích chập theo chiều sâu (depthwise convolution) dùng một kernel riêng cho mỗi kênh và không trộn các kênh với nhau, nên rất rẻ; nó chuyên bắt các đặc điểm không gian như đường viền. Tích chập 1×1 thì ngược lại: nó không nhìn sang pixel lân cận mà chỉ trộn thông tin giữa các kênh tại cùng một vị trí, thường dùng để tăng hoặc giảm số kênh.

1.4. Những linh kiện quen thuộc trong mạng hiện đại

Chuẩn hóa theo batch (Batch Normalization, viết tắt là BN) là lớp đưa các giá trị đặc trưng về cùng một thang đo bằng cách trừ đi giá trị trung bình rồi chia cho độ lệch chuẩn. Nhờ vậy, việc huấn luyện ổn định và nhanh hơn. Trong lúc huấn luyện, BN tính trung bình trên từng batch và đồng thời ghi nhớ một giá trị trung bình "chạy" (running mean) cùng phương sai "chạy" (running variance) tích lũy qua nhiều batch. Khi mô hình được dùng để dự đoán, BN dùng hai giá trị đã ghi nhớ này. Người học nên nhớ chi tiết này, vì nó là chìa khóa để hiểu sự cố với một số checkpoint của TSVM được trình bày ở Chương 7.

Kết nối tắt (residual connection) là việc cộng thẳng đầu vào của một khối vào đầu ra của nó: đầu ra bằng x cộng với $F(x)$. Ý nghĩa của nó là khối chỉ cần học phần "chỉnh sửa thêm" $F(x)$ thay vì học lại toàn bộ thông tin. Nếu khối chưa học được gì hữu ích, nó có thể cho $F(x)$ gần bằng 0 và thông tin vẫn đi qua nguyên vẹn. Kết nối tắt là lý do người ta huấn luyện được những mạng rất sâu.

Phép gộp (pooling) thu gọn một vùng nhỏ thành một giá trị duy nhất, ví dụ lấy giá trị lớn nhất trong vùng. Phép phóng to (upsampling) làm điều ngược lại, nhân đôi kích thước bản đồ đặc trưng. Phép ghép (concatenation) xếp chồng hai bản đồ đặc trưng cùng kích thước theo chiều kênh. Ba phép này là nguyên liệu chính của phần neck trong YOLO.

1.5. Học chuyển giao

Huấn luyện một mạng sâu từ đầu đòi hỏi hàng trăm nghìn ảnh có nhãn, trong khi bộ dữ liệu polyp chỉ có khoảng một nghìn ảnh. Giải pháp là học chuyển giao (transfer learning): bắt đầu từ một mô hình đã được huấn luyện sẵn trên bộ dữ liệu lớn chứa đủ loại vật thể, rồi tiếp tục huấn luyện trên dữ liệu polyp. Những kỹ năng chung như nhận ra cạnh, màu sắc và hình khối đã có sẵn, mô hình chỉ cần học thêm phần đặc thù của ảnh nội soi. Trong đề tài, cả mô hình gốc lẫn mô hình TSVM đều khởi tạo từ trọng số có sẵn yolo26s-seg.pt. Riêng khối TSVM là khối mới nên bắt đầu từ giá trị ngẫu nhiên.

1.6. Cơ chế attention và vấn đề chi phí

Tích chập có một hạn chế: mỗi điểm chỉ nhìn thấy vùng lân cận nhỏ. Muốn biết "cả bức ảnh đang nói gì", mạng phải xếp rất nhiều lớp. Cơ chế attention, nền tảng của kiến trúc Transformer, tiếp cận theo cách khác: mỗi vị trí trong ảnh được phép so sánh trực tiếp với mọi vị trí khác để quyết định nên chú ý đến đâu. Nhờ vậy, một điểm ở góc trái có thể lập tức "hỏi ý kiến" một điểm ở góc phải.

Cái giá phải trả là chi phí tính toán. Nếu có N vị trí, attention phải tính $N \times N$ cặp so sánh. Với bản đồ đặc trưng 20×20 có 400 vị trí, con số là 160.000 cặp, vẫn chấp nhận được. Nhưng với bản đồ 80×80 có 6.400 vị trí, con số đã vượt 40 triệu cặp cho mỗi ảnh. Đó là động lực để tìm một cơ chế vừa nhìn được toàn cục vừa có chi phí tăng tuyến tính theo N . Mamba, sẽ được trình bày ở Chương 4, là một câu trả lời cho bài toán này.

Ghi nhớ

CNN giỏi chi tiết cục bộ nhưng khó nắm ngữ cảnh xa. Attention nhìn được toàn ảnh nhưng chi phí tăng theo bình phương số vị trí. Mamba hướng tới nhìn toàn ảnh với chi phí tuyến tính.

CHƯƠNG 2. BÀI TOÁN PHÂN ĐOẠN THỰC THỂ

2.1. Ba mức độ hiểu một bức ảnh

Hãy hình dung bác sĩ xem một ảnh nội soi và đặt ba câu hỏi với mức độ chi tiết tăng dần. Câu hỏi thứ nhất là "Ảnh này có polyp không?"; đây là bài toán phân loại (classification), đầu ra chỉ là một nhãn cho cả ảnh. Câu hỏi thứ hai là "Polyp nằm ở đâu?"; đây là bài toán phát hiện đối tượng (object detection), đầu ra là các khung chữ nhật bao quanh từng polyp. Câu hỏi thứ ba là "Chính xác những pixel nào thuộc về polyp, và đó là polyp thứ mấy?"; đây là bài toán phân đoạn thực thể (instance segmentation), đầu ra gồm khung và mặt nạ (mask) riêng cho từng đối tượng.

Cần phân biệt phân đoạn thực thể với phân đoạn ngữ nghĩa (semantic segmentation). Phân đoạn ngữ nghĩa chỉ tô màu "pixel nào là polyp" mà không tách riêng từng polyp; nếu hai polyp nằm sát nhau, chúng hòa thành một vùng. Phân đoạn thực thể tách được polyp thứ nhất và polyp thứ hai thành hai đối tượng riêng biệt. YOLO26-seg thuộc loại phân đoạn thực thể. Các mô hình như U-Net hay PraNet thường dùng trên Kvasir-SEG lại thuộc loại phân đoạn ngữ nghĩa, vì vậy chỉ số của chúng không so sánh trực tiếp được với đề tài.

2.2. Tư tưởng của YOLO

Các phương pháp phát hiện đối tượng đời đầu làm việc theo hai giai đoạn: đầu tiên đề xuất những vùng có thể chứa vật thể, sau đó phân loại từng vùng. Cách này chính xác nhưng chậm. YOLO, viết tắt của "You Only Look Once", đưa ra ý tưởng táo bạo hơn: chỉ cho ảnh đi qua mạng một lần duy nhất và dự đoán tất cả đối tượng cùng lúc. Ảnh được chia thành lưới ô; mỗi ô chịu trách nhiệm dự đoán xem tại vị trí của nó có vật thể hay không, khung bao vật thể ra sao và vật thể thuộc lớp nào. Nhờ chỉ chạy một lần, YOLO đủ nhanh để xử lý video thời gian thực, một đặc điểm rất có giá trị với ứng dụng hỗ trợ nội soi.

Các phiên bản YOLO hiện đại là loại không dùng hộp neo (anchor-free): mỗi ô dự đoán trực tiếp khoảng cách từ tâm ô tới bốn cạnh của khung, thay vì chỉnh sửa từ một tập hộp mẫu có sẵn.

Vì nhiều ô lân cận cùng "nhìn thấy" một polyp, mô hình truyền thống thường đưa ra nhiều khung chồng lên nhau cho cùng một đối tượng. Để dọn dẹp, người ta dùng thuật toán loại bỏ không cực đại (Non-Maximum Suppression, viết tắt NMS): giữ lại khung có điểm tin cậy cao nhất và xóa những khung trùng lặp nhiều với nó. NMS là một bước hậu xử lý viết tay, tốn thời gian và phụ thuộc vào một ngưỡng phải tự chọn.

Như sẽ thấy ở Chương 3, YOLO26 được thiết kế để loại bỏ hẳn bước này.

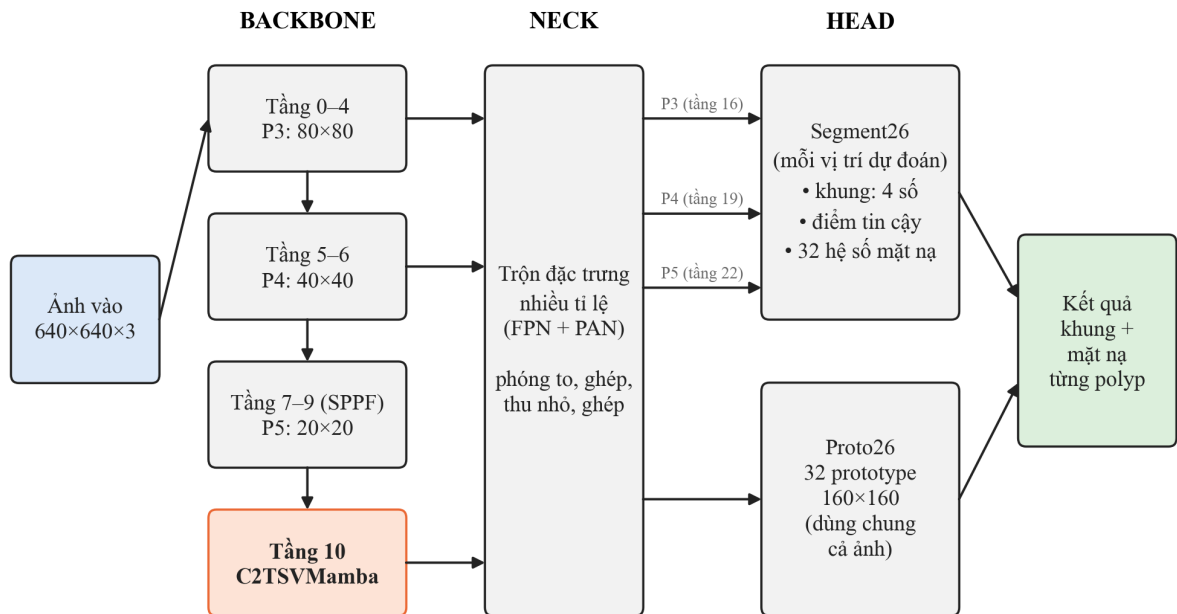
2.3. Bài toán cụ thể của đề tài

Đề tài phân đoạn polyp đại trực tràng trên ảnh nội soi, với đúng một lớp đối tượng là polyp. Bộ dữ liệu Kvasir BG20 gồm 1.000 ảnh có polyp lấy từ Kvasir-SEG và 200 ảnh niêm mạc manh tràng bình thường (normal-cecum) không có polyp. Những ảnh không có polyp này gọi là ảnh nền; chúng không phải lớp đối tượng thứ hai mà đơn giản là ảnh không có đối tượng nào. Mô hình phải học hai việc: tìm và tô đúng viền polyp khi có, và không báo nhầm khi ảnh chỉ có niêm mạc bình thường. Tập kiểm định gồm 160 ảnh, trong đó 120 ảnh có polyp chứa tổng cộng 127 polyp và 40 ảnh nền. Con số 127 này sẽ xuất hiện lại nhiều lần khi tính các chỉ số ở Chương 6. Chi tiết về cách xây dựng bộ dữ liệu được trình bày trong Bài 2.

CHƯƠNG 3. KIẾN TRÚC YOLO26-SEG

3.1. Bức tranh tổng thể

YOLO26 là thể hệ mô hình YOLO của Ultralytics; đề tài dùng phiên bản YOLO26s-seg, trong đó chữ s (small) chỉ kích cỡ nhỏ và hậu tố seg chỉ nhiệm vụ phân đoạn. Mô hình được mô tả hoàn toàn bằng một tệp cấu hình YAML, liệt kê lần lượt từng tầng: tầng nhận đầu vào từ đâu, lặp mấy lần, dùng khối gì và với tham số nào. Tệp cấu hình của đề tài là yolo26-seg-TopologyShapeVMamba.yaml. Khi đặt cạnh tệp yolo26-seg.yaml của mô hình gốc, có thể thấy hai tệp giống hệt nhau, chỉ khác đúng một dòng ở tầng 10: mô hình gốc dùng khối C2PSA, còn đề tài dùng khối C2TSVMamba. Đây là một thiết kế thí nghiệm tốt, vì mọi khác biệt về kết quả đều quy được về đúng một thay đổi. Hình 1.2 trình bày luồng xử lý tổng thể.



Hình 1.2. Kiến trúc YOLO26-seg trong đề tài; khối tô cam là vị trí đặt C2TSVMamba

3.2. Backbone và đặc trưng đa tỉ lệ

Backbone là phần trích xuất đặc trưng. Ảnh đi qua một chuỗi tầng tích chập; cứ mỗi lần gặp tầng có stride bằng 2, kích thước ảnh giảm một nửa và số kênh thường tăng lên. Ultralytics đặt tên các mức theo hệ số thu nhỏ: P3 là mức đã thu nhỏ 8 lần, P4 là 16 lần, P5 là 32 lần. Với ảnh đầu vào 640×640 , mức P3 có lưới 80×80 , P4 có lưới 40×40 và P5 có lưới 20×20 .

Có thể hình dung ba mức này như ba mức phóng đại khi xem bản đồ. Ở mức P3, mỗi ô ứng với vùng 8×8 pixel của ảnh gốc, đủ chi tiết để thấy đường viền và những polyp nhỏ. Ở mức P5, mỗi ô ứng với vùng 32×32 pixel, mất chi tiết nhưng bao quát

được bố cục của cả vùng ruột, phù hợp để nhận ra những polyp lớn và hiểu ngữ cảnh. Tổng cộng ba mức có $6.400 + 1.600 + 400 = 8.400$ vị trí, và tại mỗi vị trí mô hình đều đưa ra một dự đoán.

Các khối chính trong backbone gồm khối Conv (tích chập, chuẩn hóa batch và hàm SiLU), khối C3k2 (một khối tích chập kiểu chia nhánh rồi ghép lại, giúp học sâu mà tiết kiệm tham số), khối SPPF ở tầng 9 (gộp cục đại nhiều lần liên tiếp để mở rộng vùng nhìn) và khối ở tầng 10. Trong mô hình gốc, tầng 10 là C2PSA, một khối có cơ chế attention giúp các vị trí ở mức P5 trao đổi thông tin toàn cục. Trong đề tài, tầng 10 được thay bằng C2TSVMamba.

3.3. Neck: trộn đặc trưng nhiều tỉ lệ

Đặc trưng ở P5 hiểu ngữ cảnh tốt nhưng thô, đặc trưng ở P3 chi tiết nhưng thiếu ngữ cảnh. Neck kết hợp ưu điểm của cả hai theo hai chiều. Ở chiều đi xuống, giống mạng kim tự tháp đặc trưng FPN, đặc trưng P5 được phóng to và ghép với P4, rồi tiếp tục phóng to và ghép với P3; nhờ vậy thông tin ngữ cảnh lan xuống các mức chi tiết. Ở chiều đi lên, giống mạng PAN, kết quả ở mức P3 lại được thu nhỏ và ghép ngược lên P4 rồi P5; nhờ vậy thông tin chi tiết lan lên các mức thô. Đầu ra của neck là ba bản đồ đặc trưng đã được làm giàu ở các tầng 16 (P3), 19 (P4) và 22 (P5). Đáng chú ý là ở tầng 21, đầu ra của tầng 10 được ghép trực tiếp vào nhánh P5; đây là đường mà thông tin từ khối TSVM đi vào phần đầu ra.

3.4. Head: dự đoán khung và lớp

Head của đề tài là khối Segment26. Tại mỗi vị trí trong ba bản đồ đặc trưng, head đưa ra ba nhóm thông tin: bốn con số mô tả khung bao (khoảng cách tới bốn cạnh), một điểm tin cậy cho biết khả năng đây là polyp, và 32 hệ số mặt nạ sẽ được giải thích ngay sau đây. Một chi tiết riêng của YOLO26 là tham số `reg_max` bằng 1, nghĩa là khung được dự đoán trực tiếp bằng một giá trị cho mỗi cạnh, thay vì dự đoán một phân bố xác suất như các thế hệ trước. Vì vậy hàm mất mát DFL dùng cho phân bố đã được thay bằng hàm mất mát L1, và cột `l1_loss` xuất hiện trong tệp kết quả huấn luyện.

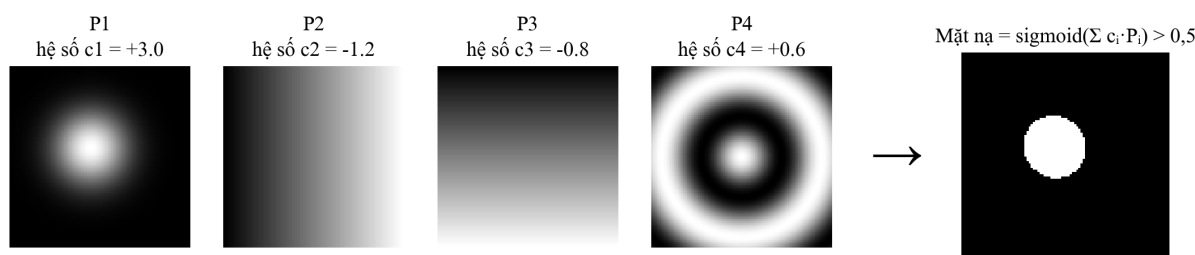
3.5. Mặt nạ được tạo từ prototype và hệ số

Đây là ý tưởng quan trọng nhất của phần "-seg". Cách đơn giản nhất để phân đoạn là cho mỗi vị trí trong 8.400 vị trí tự vẽ một mặt nạ riêng, nhưng làm vậy cực kỳ tốn kém. YOLO chọn cách khéo léo hơn, kế thừa từ phương pháp YOLACT.

Hãy hình dung ta có 32 tấm giấy can, mỗi tấm được tô sẵn một mẫu sáng tối khác nhau trải trên toàn ảnh: tấm thì sáng ở một vùng tròn, tấm thì sáng dần từ trái sang phải, tấm thì có hình vòng. Những tấm này gọi là prototype và được tạo ra một lần duy nhất cho cả bức ảnh. Với mỗi polyp, mô hình chỉ cần nói "lấy tấm thứ nhất nhân 3, tấm thứ hai nhân $-1,2$, tấm thứ ba nhân $-0,8\dots$ ", rồi chồng tất cả lên nhau. Ba mươi hai con số đó là hệ số mặt nạ. Kết quả chồng lớp được đưa qua hàm sigmoid để chuyển về khoảng từ 0 đến 1, những điểm lớn hơn 0,5 được coi là thuộc polyp, rồi mặt nạ được cắt gọn theo khung bao của đối tượng. Viết dưới dạng công thức:

$$\text{mặt nạ}_i = \text{sigmoid}(c_{i,1} \cdot P_1 + c_{i,2} \cdot P_2 + \dots + c_{i,32} \cdot P_{32})$$

Hình 1.3 minh họa ý tưởng này với bốn prototype tự tạo. Prototype thật của mô hình do mạng tự học và thường khó diễn giải bằng mắt hơn nhiều, nhưng nguyên tắc pha trộn hoàn toàn giống nhau.



Hình 1.3. Minh họa cách pha trộn prototype bằng hệ số để tạo mặt nạ (prototype tự tạo, không phải của mô hình)

Trong mã nguồn của đề tài, Segment26 dùng 32 prototype. Khởi tạo prototype là Proto26; khác với thể hệ trước chỉ dùng đặc trưng P3, Proto26 đưa đặc trưng P4 và P5 về kích thước của P3 rồi cộng lại, sau đó phóng to gấp đôi. Vì vậy với ảnh 640×640 , mỗi prototype có kích thước 160×160 . Nhận xét quan trọng rút ra từ đây là độ mịn của đường viền mặt nạ phụ thuộc chủ yếu vào chất lượng đặc trưng ở độ phân giải cao, tức mức P3.

Khi huấn luyện, Proto26 còn có thêm một nhánh phụ gọi là semseg, làm nhiệm vụ phân đoạn ngữ nghĩa cho toàn ảnh. Nhánh này đóng vai trò như bài tập bổ trợ giúp mạng học đặc trưng tốt hơn và sinh ra cột sem_loss trong tệp kết quả. Khi dự đoán, nhánh này bị bỏ đi.

Ghi nhớ

Mặt nạ của mỗi đối tượng là tổ hợp có trọng số của 32 prototype dùng chung. Head chỉ cần dự đoán 32 hệ số cho mỗi đối tượng. Prototype có kích thước 160×160 với ảnh vào 640×640 .

3.6. Học không cần NMS với hai nhánh dự đoán

YOLO26 được thiết kế theo kiểu đầu-cuối (end-to-end), trong cấu hình là tham số end2end bằng True. Ý tưởng là dạy mô hình tự đưa ra đúng một dự đoán cho mỗi đối tượng, để không cần NMS nữa. Tuy nhiên, nếu chỉ cho phép một vị trí được "nhận" mỗi polyp ngay từ đầu, tín hiệu học sẽ quá ít và mô hình học rất chậm. Giải pháp là dùng song song hai bộ đầu dự đoán có cấu trúc giống nhau.

Nhánh thứ nhất gọi là một-nhiều (one-to-many): mỗi polyp được gán cho nhiều vị trí dự đoán tốt nhất, tối đa 10 vị trí. Nhánh này cho nhiều tín hiệu học, giúp mô hình học nhanh và ổn định. Nhánh thứ hai gọi là một-một (one-to-one): mỗi polyp chỉ được gán cho đúng một vị trí. Đây là nhánh được dùng khi dự đoán thật, và vì mỗi polyp chỉ có một dự đoán nên không cần NMS.

Hàm mất mát tổng là tổng có trọng số của hai nhánh. Trọng số của nhánh một-nhiều bắt đầu ở mức 0,8 và giảm dần đều theo số epoch xuống còn 0,1; trọng số của nhánh một-một là phần còn lại để tổng luôn bằng 1:

$$\text{loss} = w \cdot \text{loss}_{\text{một-nhiều}} + (1 - w) \cdot \text{loss}_{\text{một-một}}, w \text{ giảm từ } 0,8 \text{ xuống } 0,1$$

Cách sắp xếp này rất hợp lý: đầu quá trình huấn luyện, mô hình dựa vào nhánh một-nhiều để học cho nhanh; cuối quá trình, trọng tâm dồn sang nhánh một-một vì đó là nhánh sẽ được dùng thật.

Khi chuẩn bị đưa mô hình vào sử dụng, thư viện gọi hàm fuse(). Hàm này xóa toàn bộ nhánh một-nhiều và nhánh semseg của Proto26. Mô hình nhẹ đi mà kết quả không thay đổi, vì khi dự đoán, kể cả khi đánh giá sau mỗi epoch, head vốn đã chỉ dùng nhánh một-một.

3.7. Các thành phần của hàm mất mát

Tập results.csv ghi lại giá trị của từng thành phần loss qua từng epoch, cả trên tập huấn luyện lẫn tập kiểm định. Box loss đo độ lệch giữa khung dự đoán và khung thật dựa trên một biến thể của IoU gọi là CIoU, có tính thêm khoảng cách giữa hai tâm và tỉ lệ khung. L1 loss đo sai số tuyệt đối của tọa độ khung, thay thế cho DFL như đã nói. Cls loss là hàm entropy chéo nhị phân (BCE) cho điểm tin cậy "có phải polyp không"; nó phạt cả việc bỏ sót polyp lẫn việc báo nhầm trên vùng nền. Seg loss là hàm BCE tính trên từng pixel của mặt nạ, chỉ trong phạm vi khung của đối tượng, nên phản ánh trực tiếp chất lượng đường viền. Sem loss là mất mát của nhánh phân đoạn ngữ nghĩa phụ trợ, kết hợp BCE với hệ số Dice. Trong cấu hình của đề tài, box loss và seg loss có hệ số 7,5, cls loss có hệ số 0,5.

Khi đọc tập kết quả, người học nên theo dõi hai cột val/seg_loss và metrics/mAP50-95(M). Nếu seg loss trên tập kiểm định giảm đều và mAP tăng đều, mô hình đang học tốt. Nếu seg loss trên tập huấn luyện vẫn giảm mà trên tập kiểm định lại tăng, đó là dấu hiệu quá khớp.

3.8. Cấu hình huấn luyện của đề tài

Theo tập args.yaml được lưu lại trong mỗi lần huấn luyện, cả mô hình gốc và mô hình TSVM đều được huấn luyện 100 epoch, batch 8, ảnh kích thước 640, bộ tối ưu AdamW với tốc độ học ban đầu 0,001, khởi tạo từ trọng số yolo26s-seg.pt, trên GPU Tesla T4 của Kaggle. So sánh hai tập args.yaml của seed 0 cho thấy chúng chỉ khác nhau ở các dòng khai báo mô hình, cách ghi trọng số khởi tạo, tên lần chạy và thư mục lưu; mọi tham số dữ liệu, tăng cường dữ liệu và huấn luyện đều trùng nhau. Mã nguồn YOLO26 có sẵn một bộ tối ưu mới tên MuSGD, nhưng đề tài không dùng nó. Theo công bố của Ultralytics, YOLO26 còn có các cải tiến về hàm mất mát và cách gán nhãn cho vật thể nhỏ; giáo trình chưa đối chiếu chi tiết những cải tiến này trong mã nguồn nên không trình bày sâu.

CHƯƠNG 4. TỪ MÔ HÌNH KHÔNG GIAN TRẠNG THÁI ĐẾN VMAMBA

4.1. Ý tưởng của mô hình không gian trạng thái

Hãy hình dung việc đọc một cuốn sách dày. Ta không thể nhớ từng chữ của mọi trang, nhưng có thể giữ một cuốn sổ ghi chú. Mỗi khi lật sang trang mới, ta làm hai việc: gạch bớt những ghi chú cũ không còn quan trọng, và ghi thêm những điều đáng nhớ từ trang đang đọc. Khi cần trả lời một câu hỏi, ta chỉ việc tra cuốn sổ. Cuốn sổ ấy là một bản tóm tắt gọn của mọi thứ đã đọc, và kích thước của nó không phình to theo số trang.

Mô hình không gian trạng thái (State Space Model, viết tắt SSM) hoạt động đúng như vậy. Nó đọc một chuỗi dữ liệu $x_1, x_2, \dots, x_L$ từng phần tử một và duy trì một trạng thái ẩn h đóng vai trò cuốn sổ. Ở dạng rời rạc, mỗi bước gồm hai phương trình:

$$\begin{aligned}h_t &= a_t \cdot h_{t-1} + b_t \\ y_t &= C_t \cdot h_t\end{aligned}$$

Phương trình thứ nhất cập nhật trí nhớ: hệ số a_t nằm trong khoảng từ 0 đến 1 quyết định giữ lại bao nhiêu phần trí nhớ cũ (gần 1 là nhớ lâu, gần 0 là quên nhanh), còn b_t là lượng thông tin mới được ghi thêm từ phần tử x_t . Phương trình thứ hai đọc ra kết quả từ trí nhớ. Vì mỗi bước chỉ tốn một lượng tính toán cố định, chi phí xử lý cả chuỗi tăng tuyến tính theo độ dài L , khác hẳn với attention có chi phí tăng theo L^2 .

4.2. Rời rạc hóa và bước thời gian

Về mặt lý thuyết, SSM xuất phát từ một hệ phương trình vi phân liên tục với ma trận trạng thái A âm. Để dùng cho dữ liệu rời rạc như chuỗi pixel, người ta rời rạc hóa hệ này với một "bước thời gian" Δ . Kết quả là hệ số quên được tính theo công thức $a = \exp(\Delta \cdot A)$. Vì A âm và Δ dương, a luôn nằm trong khoảng từ 0 đến 1. Có thể hiểu Δ như mức độ "chú tâm" vào phần tử hiện tại: Δ lớn thì a nhỏ, mô hình quên nhanh quá khứ và tập trung vào hiện tại; Δ nhỏ thì a gần 1, mô hình giữ trí nhớ lâu và gần như lướt qua phần tử hiện tại. Lượng thông tin ghi thêm cũng tỉ lệ với Δ : $b = \Delta \cdot B \cdot x$.

4.3. Tính chọn lọc: điểm mới của Mamba

Ở SSM cổ điển, các hệ số Δ , B và C cố định, áp dụng như nhau cho mọi phần tử. Điều này giống một người đọc sách mà trang nào cũng ghi chép với cùng một mức độ, dù trang đó quan trọng hay không. Mamba đưa ra cải tiến then chốt: Δ , B và C được tính ra từ chính dữ liệu đang đọc. Gặp một vùng ảnh có dấu hiệu bất thường, mô hình có thể tăng Δ để ghi mạnh vào trí nhớ; gặp vùng niềm mạc bình thường, mô hình có thể giảm Δ để lướt qua. Khả năng tự quyết định nên nhớ gì và quên gì theo nội dung được gọi là tính chọn lọc (selective), và cơ chế này được gọi là quét chọn lọc (selective scan).

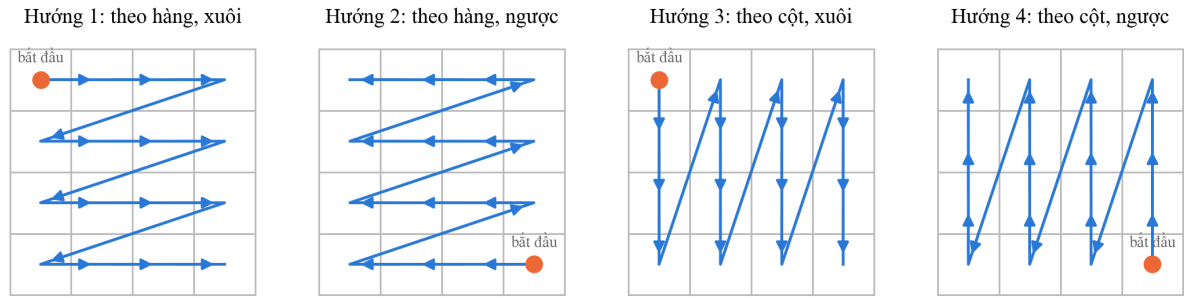
4.4. Tính toán song song

Nhìn công thức $h_t = a_t \cdot h_{t-1} + b_t$, ta dễ nghĩ phải tính tuần tự từng bước, vì muốn có h_t phải có h_{t-1} . Điều đó sẽ rất chậm trên GPU, vốn mạnh ở tính toán song song. May mắn là phép kết hợp hai bước liên tiếp có tính kết hợp: áp dụng bước (a_1, b_1) rồi bước (a_2, b_2) tương đương với một bước duy nhất $(a_1 a_2, a_2 b_1 + b_2)$. Nhờ tính chất này, ta có thể gộp từng cặp bước, rồi gộp từng cặp kết quả, và cứ thế tiếp tục, giống như thi đấu loại trực tiếp. Toàn bộ chuỗi dài L được tính xong trong khoảng $\log_2 L$ vòng song song. Trong mã nguồn của đề tài, hàm `parallel_associative_scan` thực hiện đúng ý tưởng này bằng PyTorch thuần.

4.5. VMamba và phép quét bốn hướng

Mamba được thiết kế cho chuỗi một chiều như văn bản, còn ảnh là dữ liệu hai chiều. Cách đơn giản nhất là trải ảnh thành một hàng dài theo thứ tự từng dòng, giống đọc sách từ trái sang phải, từ trên xuống dưới. Nhưng như vậy, khi đọc đến điểm ở đầu ảnh, mô hình chưa biết gì về phần cuối ảnh; ngoài ra hai điểm nằm sát nhau theo chiều dọc lại cách nhau cả một hàng trong chuỗi.

VMamba (Vision Mamba) giải quyết vấn đề này bằng khối SS2D (2D Selective Scan), quét ảnh theo bốn hướng: theo hàng từ đầu đến cuối, theo hàng từ cuối về đầu, theo cột từ đầu đến cuối và theo cột từ cuối về đầu, như minh họa ở Hình 1.4. Mỗi hướng được xử lý bằng một bộ quét chọn lọc riêng, sau đó bốn kết quả được đưa về lại lưới hai chiều và cộng với nhau. Sau bốn lượt quét, mỗi điểm đã nhận được thông tin từ mọi phía của ảnh.



Hình 1.4. Bốn thứ tự quét của khối SS2D trên một lưới 4×4

Trình tự xử lý của SS2D trong mã nguồn của đề tài như sau. Đầu tiên, đặc trưng vào được chiếu thành hai phần là u và z . Phần u đi qua một lớp tích chập theo chiều sâu 3×3 và hàm SiLU để trộn thông tin lân cận trước khi quét. Từ u , mô hình tạo ra bốn chuỗi theo bốn hướng. Với mỗi chuỗi, các lớp chiếu tuyến tính tính ra Δ , B và C phụ thuộc vào dữ liệu; hệ số quên được tính bằng $a = \exp(\Delta \cdot A)$; sau đó phép quét song song tính toàn bộ trạng thái ẩn và đọc ra kết quả. Bốn kết quả được gập về dạng lưới và cộng lại. Cuối cùng, kết quả được nhân với $\text{SiLU}(z)$, đóng vai trò một cái công điều tiết lượng thông tin đi ra, rồi chiếu về số kênh ban đầu.

Cần lưu ý rằng SS2D trong đề tài là một bản cài đặt bằng PyTorch thuần theo ý tưởng của VMamba, không dùng nhân CUDA tối ưu của tác giả gốc. Vì vậy khi viết khóa luận, nên mô tả là "triển khai SS2D theo ý tưởng VMamba" và không nên khẳng định tốc độ tương đương bản gốc nếu chưa đo.

Ghi nhớ

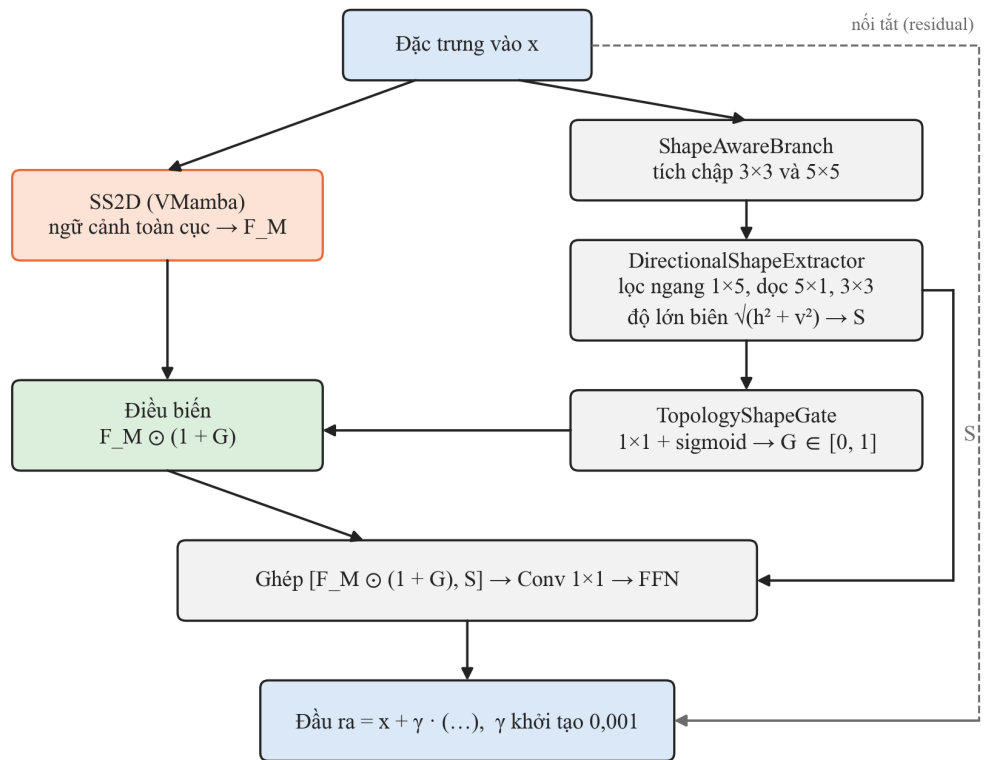
SSM giữ một trí nhớ được cập nhật tuần tự. Mamba làm cho việc ghi nhớ trở nên chọn lọc theo nội dung. VMamba quét ảnh theo bốn hướng để mỗi điểm nhận được thông tin từ toàn ảnh, với chi phí tuyến tính.

CHƯƠNG 5. KHỐI TSVM TRONG ĐỀ TÀI

5.1. Động cơ và vị trí đặt khối

Polyp có những đặc điểm hình học khá rõ: thường có dạng tròn hoặc bầu dục, đường viền liền mạch và khép kín. VMamba giỏi nắm bắt ngữ cảnh toàn cục, nhưng bản thân nó không chuyên về đường viền. Ý tưởng của khối TSVM (Topology-Shape-aware VMamba) là ghép VMamba với một nhánh chuyên trích xuất thông tin hình dạng và biên, rồi để thông tin hình dạng "dẫn đường" cho đặc trưng toàn cục.

Khối được đặt ở tầng 10, đỉnh của backbone, thay cho khối attention C2PSA. Tại đây bản đồ đặc trưng có kích thước 20×20 , tức chuỗi quét dài $L = 400$ phần tử. Với phiên bản s, tầng này có 512 kênh; vỏ bọc C2TSVMamba chia đôi số kênh nên khối TSVMamba bên trong làm việc với 256 kênh và trạng thái ẩn có kích thước 16. Hình 1.5 mô tả cấu trúc của khối.



Hình 1.5. Cấu trúc khối TSVMamba trong mã nguồn của đề tài

5.2. Nhánh toàn cục

Nhánh thứ nhất chính là khối SS2D đã trình bày ở Chương 4. Nó nhận đặc trưng vào x và cho ra đặc trưng toàn cục, ký hiệu F_M , trong đó mỗi vị trí đã tổng hợp thông tin từ cả bản đồ 20×20 .

5.3. Nhánh hình dạng và biên có hướng

Nhánh thứ hai gồm hai bước. Bước đầu, khối ShapeAwareBranch dùng hai lớp tích chập theo chiều sâu với kernel 3×3 và 5×5 , giống như hai chiếc kính lúp có độ phóng đại khác nhau, để bắt các đường cong và đường viền cục bộ ở hai tầm nhìn; kết quả của hai lớp được ghép lại và trộn bằng tích chập 1×1 .

Bước tiếp theo, khối DirectionalShapeExtractor dùng ba bộ lọc có hướng. Bộ lọc dạng dải 1×5 nhảy với những đường viền chạy theo chiều ngang, bộ lọc 5×1 nhảy với những đường viền chạy theo chiều dọc, còn bộ lọc 3×3 bắt độ cong cục bộ theo mọi hướng. Từ hai phản hồi ngang h và dọc v , khối tính thêm độ lớn biên theo công thức $\sqrt{(h^2 + v^2)}$, tương tự cách bộ lọc Sobel cổ điển tính độ lớn gradient của ảnh: chỗ nào có biên mạnh theo bất kỳ hướng nào thì giá trị lớn. Bốn thành phần này được ghép lại, trộn với đặc trưng hình dạng ban đầu, tạo thành đặc trưng cấu trúc ký hiệu S .

5.4. Cổng điều biến

Từ S , khối TopologyShapeGate dùng một lớp tích chập 1×1 và hàm sigmoid để tạo ra một bản đồ G có giá trị từ 0 đến 1 tại mỗi vị trí và mỗi kênh. Bản đồ G được dùng để điều biến đặc trưng toàn cục theo công thức:

$$F'_M = F_M \odot (1 + G)$$

trong đó dấu $\odot$ là phép nhân từng phần tử. Tại những vị trí có dấu hiệu hình dạng rõ (G gần 1), đặc trưng toàn cục được khuếch đại lên tới gấp đôi; tại những vị trí khác (G gần 0), đặc trưng được giữ nguyên. Việc dùng $(1 + G)$ thay vì chỉ G là có chủ ý: nếu nhân với G , những vùng có G gần 0 sẽ bị xóa sạch thông tin toàn cục; nhân với $(1 + G)$ thì thông tin hình dạng chỉ được phép làm nổi bật, không được phép tắt hẳn đặc trưng toàn cục.

5.5. Hợp nhất, tinh chỉnh và kết nối tắt

Đặc trưng toàn cục đã điều biến được ghép với đặc trưng cấu trúc S , đi qua một lớp tích chập 1×1 để hợp nhất, rồi qua một mạng truyền thẳng nhỏ (FFN) gồm hai lớp 1×1 để tinh chỉnh. Cuối cùng, kết quả được nhân với một hệ số học được γ và cộng vào đầu vào theo kiểu kết nối tắt:

$$\text{đầu ra} = x + \gamma \cdot \text{FFN}(\text{hợp nhất}[F'_M, S])$$

Hệ số γ được khởi tạo bằng 0,001. Đây là một mẹo ổn định quan trọng. Vì mọi tầng khác của mô hình đã được khởi tạo từ trọng số học sẵn, còn khối TSVM thì mới tinh, nếu để khối mới tác động mạnh ngay từ đầu, nó có thể làm rối loạn những đặc

trung tốt có sẵn. Với γ rất nhỏ, lúc đầu khối gần như "trong suốt", đầu ra xấp xỉ bằng đầu vào; theo thời gian, γ được học tăng dần khi khối đã thực sự hữu ích.

5.6. Vỏ bọc C2TSVMamba

Để cắm khối mới vào đúng chỗ của C2PSA mà không phải thay đổi các tầng xung quanh, đề tài dùng vỏ bọc C2TSVMamba theo mẫu thiết kế quen thuộc của Ultralytics. Đầu vào đi qua một lớp tích chập 1×1 rồi được chia đôi theo kênh. Một nửa giữ nguyên, nửa còn lại đi qua các khối TSVMamba (cấu hình dùng 2 khối). Hai nửa được ghép lại và đi qua một lớp 1×1 nữa để trở về đúng số kênh ban đầu. Cách chia đôi này vừa tiết kiệm tính toán vừa giữ một đường đi thẳng cho thông tin.

5.7. Các chế độ thí nghiệm loại bỏ

Mã nguồn của khối hỗ trợ bốn chế độ: đầy đủ (VMamba, nhánh hình dạng và cổng điều biến), chỉ VMamba, chỉ nhánh hình dạng, và VMamba cùng nhánh hình dạng nhưng không có cổng điều biến. Các chế độ này phục vụ thí nghiệm loại bỏ (ablation study), tức là lần lượt bỏ từng thành phần để xem thành phần đó đóng góp bao nhiêu. Tại thời điểm biên soạn, kho mã chưa có kết quả 10 seed cho từng chế độ, nên đề tài chưa thể khẳng định đóng góp riêng của từng nhánh.

5.8. Nhìn nhận một cách phản biện

Người học nên chuẩn bị sẵn ba nhận xét phản biện. Thứ nhất, tên gọi "nhận biết cấu trúc topo" cần được dùng thận trọng. Về bản chất, nhánh này gồm các tích chập có hướng và độ lớn biến; mô hình không có ràng buộc topo tường minh nào, chẳng hạn như hàm mất mát đếm số vùng liên thông. Cách viết chính xác là "khối trích đặc trưng hình dạng và biên có hướng, được đặt tên là topology-shape-aware".

Thứ hai, khối được đặt ở mức P5 có độ phân giải 20×20 , nơi mỗi ô ứng với vùng 32×32 pixel của ảnh gốc, nên thông tin đường viền chi tiết đã bị nén đáng kể. Trong khi đó, mặt nạ chủ yếu được tạo từ prototype có độ phân giải cao gần với mức P3. Điều này giải thích vì sao có ý tưởng đặt khối VMamba ở mức P3; tuy nhiên hướng này hiện chưa có mã nguồn trong kho mã và chưa được kiểm chứng.

Thứ ba, ở kích thước 20×20 , attention vốn đã khá rẻ, và SS2D cài bằng PyTorch thuần có thể còn chậm hơn. Vì vậy không nên lập luận rằng TSVM giúp mô hình nhanh hơn nếu chưa đo đạc cụ thể.

Ghi nhớ

TSVM thay khối attention ở tầng 10 bằng tổ hợp: VMamba lấy ngữ cảnh toàn cục, nhánh hình dạng lấy biên có hướng, cộng $(1 + G)$ cho hình dạng khuếch đại đặc trưng toàn cục, và hệ số γ nhỏ giúp khối mới hòa nhập ổn định.

CHƯƠNG 6. CÁC CHỈ SỐ ĐÁNH GIÁ

6.1. IoU: thước đo độ trùng khớp

Mọi chỉ số đánh giá trong phát hiện và phân đoạn đều xây dựng trên một khái niệm gốc là IoU (Intersection over Union), tức tỉ số giữa phần giao và phần hợp của vùng dự đoán với vùng thật:

$$\text{IoU} = \text{diện tích phần giao} / \text{diện tích phần hợp}$$

IoU bằng 1 khi hai vùng trùng khít và bằng 0 khi chúng không chạm nhau. Ví dụ, mặt nạ thật có 1.000 pixel, mặt nạ dự đoán có 900 pixel và hai mặt nạ trùng nhau 800 pixel. Phần hợp là $1.000 + 900 - 800 = 1.100$ pixel, nên $\text{IoU} = 800 / 1.100 \approx 0,727$.

IoU có hai phiên bản. Box IoU tính trên hai khung chữ nhật; mask IoU tính trên từng pixel của hai mặt nạ. Mask IoU khắt khe hơn nhiều với đường viền: hai khung có thể trùng gần khít trong khi viền mặt nạ bên trong vẫn lệch nhau đáng kể.

6.2. Khi nào một dự đoán được coi là đúng

Để chấm một dự đoán, ta chọn trước một ngưỡng IoU, chẳng hạn 0,5. Một dự đoán khớp được với một polyp thật với IoU từ ngưỡng trở lên được gọi là dương tính thật (True Positive, TP). Một dự đoán không khớp được với polyp nào, hoặc trùng lặp với một dự đoán khác đã khớp, là dương tính giả (False Positive, FP); về mặt lâm sàng, đó là một lần báo động giả. Một polyp thật mà không có dự đoán nào khớp là âm tính giả (False Negative, FN), tức bỏ sót polyp, trường hợp nguy hiểm nhất trong thực hành nội soi.

Trong phân loại ảnh còn có âm tính thật (True Negative, TN), nghĩa là "không báo gì và đúng là không có gì". Nhưng trong phát hiện đối tượng, khái niệm này không đếm được một cách tự nhiên, vì mỗi ảnh có vô số vùng trống mà mô hình đúng khi không báo. Thư viện Ultralytics cũng không tự đếm TN. Điều này sẽ được nhắc lại khi đọc ma trận nhầm lẫn ở Chương 7.

6.3. Precision và Recall

Từ TP, FP và FN, ta có hai chỉ số cơ bản:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad \text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Precision (độ chính xác) trả lời câu hỏi: trong những gì mô hình báo là polyp, bao nhiêu phần trăm đúng là polyp? Recall (độ phủ) trả lời câu hỏi: trong tất cả polyp thật, mô hình tìm được bao nhiêu phần trăm? Lấy ví dụ tập kiểm định có 127 polyp,

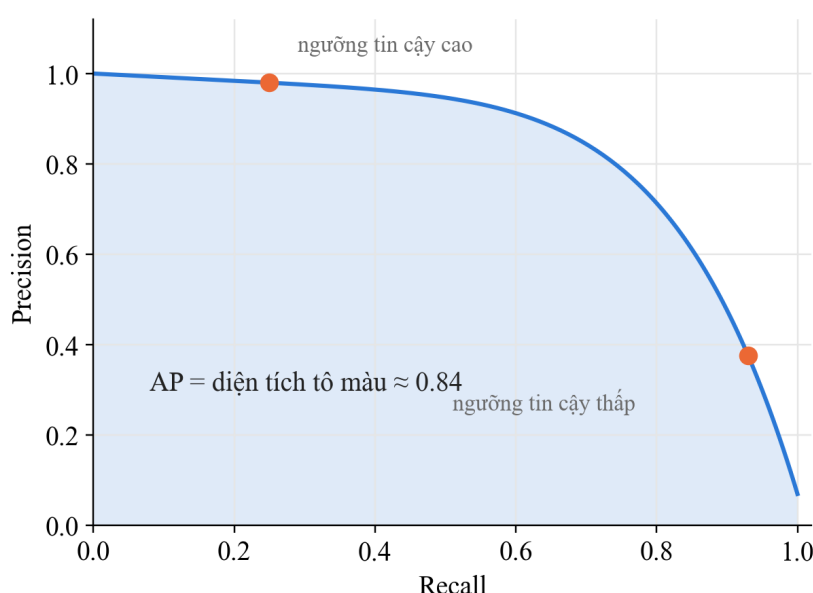
mô hình tìm đúng 111, bỏ sót 16 và báo nhảm 18 lần. Khi đó $\text{Precision} = 111 / 129 \approx 0,860$ và $\text{Recall} = 111 / 127 \approx 0,874$.

Hai chỉ số này thường kéo nhau theo hai hướng ngược chiều. Mỗi dự đoán đi kèm một điểm tin cậy từ 0 đến 1, và ta chỉ giữ những dự đoán có điểm vượt một ngưỡng tin cậy nào đó. Hạ ngưỡng xuống, mô hình báo nhiều hơn, tìm được nhiều polyp hơn nên Recall tăng, nhưng cũng báo nhảm nhiều hơn nên Precision giảm. Nâng ngưỡng lên thì ngược lại. Trong y tế, người ta thường ưu tiên Recall để không bỏ sót bệnh, nhưng Precision quá thấp sẽ khiến bác sĩ mệt mỏi vì liên tục bị báo động giả. Chỉ số F1, bằng $2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$, là trung bình điều hòa giúp cân bằng hai mặt này.

Cần cảnh giác với việc chỉ nhìn một chỉ số. Một mô hình rất dè dặt, chỉ báo khi cực kỳ chắc chắn, có thể đạt Precision gần 1 nhưng bỏ sót rất nhiều polyp. Vì vậy cần một chỉ số đánh giá mô hình trên mọi mức ngưỡng tin cậy, và đó là AP.

6.4. Đường Precision–Recall và AP

Thay vì chọn một ngưỡng tin cậy cố định, ta quét qua mọi ngưỡng từ cao xuống thấp. Mỗi ngưỡng cho một cặp giá trị Precision và Recall, tương ứng một điểm trên mặt phẳng. Nối các điểm lại, ta được đường Precision–Recall như Hình 1.7. Ở ngưỡng cao, mô hình chỉ báo vài dự đoán chắc chắn nhất, nên Precision cao nhưng Recall thấp. Khi hạ ngưỡng dần, đường đi sang phải và thường đi xuống.



Hình 1.6. Đường Precision–Recall minh họa; AP là diện tích phần tô màu bên dưới đường

Độ chính xác trung bình (Average Precision, AP) là diện tích nằm dưới đường này, có giá trị từ 0 đến 1. Một mô hình lý tưởng vừa tìm được hết polyp vừa không báo

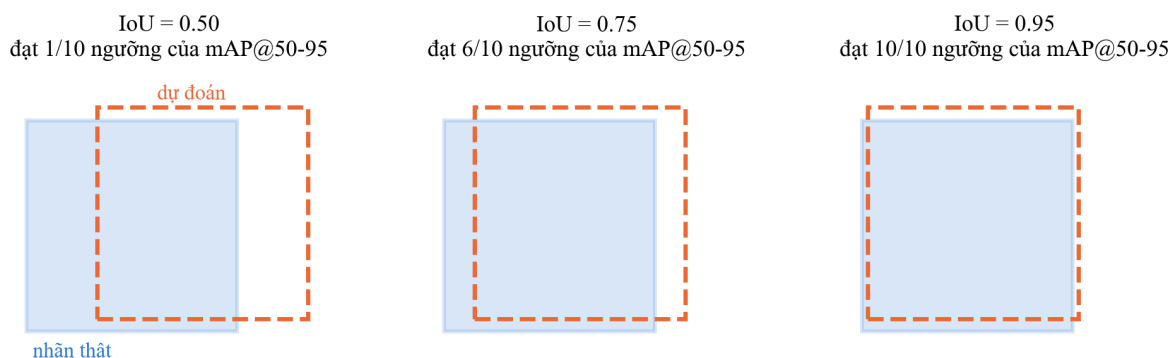
nhằm sẽ có đường nằm sát mép trên và AP gần bằng 1. AP tóm gọn chất lượng của mô hình trên mọi ngưỡng tin cậy vào một con số duy nhất. Thư viện của đề tài tính diện tích này theo cách nội suy 101 điểm của bộ chuẩn COCO.

6.5. mAP@50 và mAP@50-95

Chữ m trong mAP là viết tắt của mean, nghĩa là lấy trung bình AP qua các lớp đối tượng. Đề tài chỉ có một lớp là polyp, nên mAP chính là AP của lớp polyp. Điều làm nên sự khác biệt giữa các biến thể mAP là ngưỡng IoU được dùng để quyết định một dự đoán là đúng hay sai.

mAP@50 dùng một ngưỡng IoU duy nhất là 0,5: dự đoán chỉ cần trùng khoảng một nửa với vùng thật là được tính đúng. Chỉ số này trả lời câu hỏi "mô hình có tìm ra polyp không?". mAP@50-95 tính AP lần lượt ở mười ngưỡng IoU 0,50; 0,55; 0,60; ...; 0,95 rồi lấy trung bình. Muốn đạt điểm cao, dự đoán phải khớp với vùng thật ở cả những ngưỡng rất khắt khe. Chỉ số này trả lời câu hỏi "mô hình tô viền polyp chính xác đến đâu?".

Hình 1.6 cho thấy mức độ khắt khe đó. Một dự đoán có IoU bằng 0,5 chỉ được tính đúng ở 1 trong 10 ngưỡng; dự đoán có IoU bằng 0,75 được tính đúng ở 6 ngưỡng; phải đạt IoU 0,95, gần như trùng khít, mới được tính đúng ở cả 10 ngưỡng.



Hình 1.7. Ba mức IoU và số ngưỡng của mAP@50-95 mà mỗi mức vượt qua

Khi áp dụng cho mặt nạ, ta có Mask mAP; khi áp dụng cho khung, ta có Box mAP. Trong tệp results.csv, ký hiệu (M) chỉ mặt nạ và (B) chỉ khung. Đề tài chọn **Mask mAP@50-95** làm chỉ số chính để so sánh mô hình, vì nó phản ánh đúng nhất chất lượng phân đoạn đường viền.

Ghi nhớ

Mask mAP@50 của đề tài vào khoảng 0,91 trong khi Mask mAP@50-95 chỉ vào khoảng 0,72. Khoảng cách đó cho biết mô hình tìm polyp rất tốt nhưng đường viền chưa thật sát ở các ngưỡng IoU cao. Muốn cải thiện chỉ số chính, phải cải thiện chất lượng đường viền.

6.6. Hệ số Dice và các bài báo phân đoạn polyp

Nhiều công trình về Kvasir-SEG dùng mô hình phân đoạn ngữ nghĩa và báo cáo hệ số Dice và IoU trung bình theo pixel, thay vì mAP. Hệ số Dice được định nghĩa là 2 lần phần giao chia cho tổng diện tích hai vùng, và có quan hệ chặt chẽ với IoU:

$$\text{Dice} = 2 \cdot |A \cap B| / (|A| + |B|) = 2 \cdot \text{IoU} / (1 + \text{IoU})$$

Chẳng hạn IoU bằng 0,8 thì Dice xấp xỉ 0,889. Quy trình đánh giá YOLO của đề tài không xuất hệ số Dice. Vì đây là hai thước đo khác nhau, lại thường được tính trên những cách chia dữ liệu khác nhau, người học không nên đặt Mask mAP@50-95 của đề tài cạnh con số "Dice 0,9x" của một bài báo khác rồi kết luận mô hình nào tốt hơn.

6.7. Đánh giá trên nhiều seed

Quá trình huấn luyện chứa nhiều yếu tố ngẫu nhiên: giá trị khởi tạo của khối mới, thứ tự các ảnh trong từng batch, các phép tăng cường dữ liệu. Hạt giống ngẫu nhiên (seed) là con số khởi động bộ sinh số ngẫu nhiên; cùng seed thì cùng chuỗi ngẫu nhiên. Đổi seed giống như tung lại xúc xắc: cùng một mô hình, cùng một dữ liệu, nhưng kết quả có thể dao động. Nếu chỉ huấn luyện một lần rồi so sánh, ta không biết chênh lệch đến từ mô hình hay chỉ do may rủi. Vì vậy đề tài huấn luyện mỗi mô hình 10 lần với seed từ 0 đến 9.

Từ 10 kết quả, ta tính giá trị trung bình để biết mức hiệu năng điển hình và độ lệch chuẩn để biết mức dao động; độ lệch chuẩn nhỏ nghĩa là mô hình ổn định, ít phụ thuộc vào may rủi. Để so sánh hai mô hình, đề tài dùng kiểm định t ghép cặp (paired t-test): với mỗi seed, lấy kết quả của TSVM trừ kết quả của mô hình gốc cùng seed, rồi kiểm tra xem trung bình của mười hiệu số này có khác 0 một cách đáng tin cậy hay không. Ghép cặp theo seed giúp loại bớt phần dao động chung của từng lần chạy.

Kết quả của kiểm định là giá trị p (p-value). Có thể hiểu p như sau: giả sử thật ra hai mô hình tốt ngang nhau, thì xác suất ta vẫn quan sát được một chênh lệch lớn cỡ này, chỉ do ngẫu nhiên, là bao nhiêu? Nếu p nhỏ hơn 0,05, theo quy ước phổ biến, ta

nói chênh lệch có ý nghĩa thống kê. Một hiểu lầm hay gặp là coi $p = 0,38$ nghĩa là "38% khả năng TSVM tốt hơn". Điều đó không đúng: p không phải xác suất để một giả thuyết đúng, nó chỉ cho biết dữ liệu hiện có chưa đủ mạnh để bác bỏ khả năng hai mô hình như nhau.

Ngoài ra, đề tài còn dùng tỉ số phương sai để so độ ổn định giữa hai mô hình và đếm số seed mà mỗi mô hình thắng. Những con số này có giá trị tham khảo nhưng không thay thế được kiểm định thống kê.

6.8. Chọn epoch tốt nhất

Sau mỗi epoch, mô hình được đánh giá trên tập kiểm định và kết quả được ghi vào results.csv. Gói phân tích 10 seed của đề tài lấy số liệu tại epoch có Mask mAP@50-95 cao nhất. Trong khi đó, thư viện Ultralytics lưu tệp trọng số best.pt theo một tiêu chí hơi khác: tổng của Box mAP@50-95 và Mask mAP@50-95. Hai tiêu chí thường cho cùng một epoch nhưng không bắt buộc phải trùng. Vì vậy, khi đánh giá lại từ tệp best.pt, người học cần kiểm tra xem tệp đó ứng với epoch nào trước khi so số liệu.

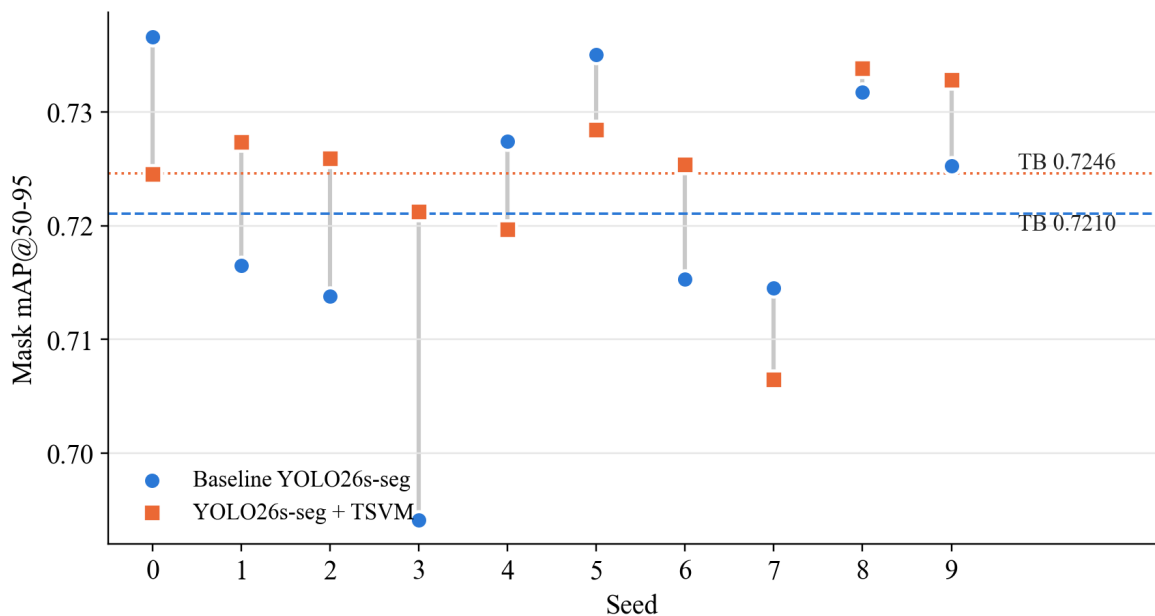
CHƯƠNG 7. ĐỌC KẾT QUẢ THỰC NGHIỆM CỦA ĐỀ TÀI

7.1. Kết quả 10 seed

Bảng dưới đây trích từ báo cáo tổng hợp của gói phân tích 10 seed, với số liệu gốc nằm trong tệp `raw_10seeds_extracted_metrics.csv`. Mỗi ô ghi giá trị trung bình cộng trừ độ lệch chuẩn qua 10 seed.

Chỉ số	Mô hình gốc	Mô hình TSVM	Giá trị p
Mask mAP@50-95	$0,7210 \pm 0,0129$	$0,7246 \pm 0,0078$	0,3839
Mask mAP@50	$0,9119 \pm 0,0107$	$0,9062 \pm 0,0082$	0,2273
Mask Precision	$0,9023 \pm 0,0339$	$0,9118 \pm 0,0246$	0,5428
Mask Recall	$0,8584 \pm 0,0252$	$0,8625 \pm 0,0173$	0,5907
Val Seg Loss	$1,3045 \pm 0,0867$	$1,2424 \pm 0,0387$	0,0908

Hình 1.8 biểu diễn chỉ số chính theo từng seed. Mỗi cặp điểm cùng cột là hai mô hình huấn luyện với cùng một seed; hai đường ngang là giá trị trung bình của mỗi mô hình.



Hình 1.8. Mask mAP@50-95 của mô hình gốc và mô hình TSVM theo từng seed

7.2. Diễn giải đúng mức

Điều đầu tiên đọc được là Mask mAP@50-95 trung bình của TSVM cao hơn mô hình gốc 0,0036. Tuy nhiên giá trị p bằng 0,3839, lớn hơn nhiều so với 0,05, nên chênh lệch này chưa có ý nghĩa thống kê. Nhìn Hình 1.8 sẽ thấy rõ lý do: TSVM thắng ở 6 seed nhưng thua ở 4 seed, và các khoảng chênh lệch có lúc lớn lúc nhỏ. Vì vậy không được viết rằng TSVM "cải thiện đáng kể" so với mô hình gốc.

Điểm tích cực có thể nêu là độ lệch chuẩn của TSVM nhỏ hơn rõ rệt: 0,0078 so với 0,0129. Trên hình, các điểm vuông của TSVM tụ lại gần nhau hơn, còn các điểm tròn của mô hình gốc trải rộng hơn, thấp nhất ở seed 3 và cao nhất ở seed 0. Như vậy TSVM có xu hướng ổn định hơn trước tác động của yếu tố ngẫu nhiên. Loss phân đoạn trên tập kiểm định của TSVM cũng thấp hơn ở 8 trên 10 seed, với p bằng 0,0908, gần ngưỡng nhưng vẫn chưa đạt. Ngược lại, Mask mAP@50 của TSVM lại thấp hơn một chút. Nói cách khác, không có sự cải thiện đồng loạt trên mọi chỉ số.

Một cách diễn đạt phù hợp khi viết khóa luận hoặc khi bảo vệ là: "Trên 10 seed, TSVM đạt Mask mAP@50-95 trung bình 0,7246 so với 0,7210 của mô hình gốc, với độ lệch chuẩn thấp hơn; tuy nhiên kiểm định t ghép cặp cho $p = 0,3839$, nên chênh lệch chưa có ý nghĩa thống kê ở mức $\alpha = 0,05$."

7.3. Về ma trận nhầm lẫn

Gói phân tích có tệp ma trận nhầm lẫn với bốn cột TP, FN, FP và TN cho từng seed. Ví dụ với mô hình gốc ở seed 0, các giá trị lần lượt là 111, 16, 18 và 22. Có thể kiểm tra rằng $TP + FN = 127$, đúng bằng số polyp thật, và $FP + TN = 40$, đúng bằng số ảnh nền. Tuy nhiên như đã nói ở mục 6.2, Ultralytics không tự đếm TN. Báo cáo kiểm toán của đề tài xác nhận TN trong tệp này là số được suy ra bằng 40 trừ FP chứ không phải số đo, và các con số FP, TN đang có chênh lệch giữa ảnh biểu đồ và tệp số liệu. Vì vậy, các giá trị FP và TN này không nên được dùng làm số liệu chính trong khóa luận cho đến khi được tái lập.

7.4. Sự cố checkpoint của seed 0, 5 và 8

Khi đánh giá lại các tệp trọng số đã lưu của TSVM ở seed 0, 5 và 8, kết quả sụp đổ hoàn toàn; chẳng hạn seed 5 không tìm đúng được polyp nào, dù số liệu ghi trong results.csv lúc huấn luyện vẫn rất tốt. Hồ sơ kỹ thuật cũ cho rằng nguyên nhân là hàm fuse(), nhưng báo cáo điều tra mới nhất của nhóm đã chỉ ra nguyên nhân thật.

Bên trong khối TSVM, nhánh SS2D khuếch đại tín hiệu lên rất lớn, tới hàng triệu, khiến lớp chuẩn hóa batch ngay sau nó phải ghi nhớ giá trị trung bình chạy cỡ

hàng triệu. Mã nguồn cũ lưu checkpoint ở định dạng số thực nửa độ chính xác FP16, vốn chỉ biểu diễn được giá trị tối đa khoảng 65.504. Giá trị trung bình bị cắt về 65.504, nên khi mở tệp ra dùng, lớp chuẩn hóa trừ sai và toàn bộ dự đoán hỏng. Thí nghiệm hiệu chỉnh lại riêng thống kê của lớp chuẩn hóa đã khôi phục Mask mAP50 của seed 5 từ 0 lên 0,9099, khớp với giá trị 0,9096 ghi nhận lúc huấn luyện, qua đó khẳng định quan hệ nhân quả. Phiên bản quy trình mới của nhóm đã chuyển sang lưu checkpoint ở định dạng FP32 để khắc phục.

Bài học rút ra gồm hai điều. Thứ nhất, số liệu 10 seed được lấy từ results.csv, đo trong lúc huấn luyện, nên không bị ảnh hưởng bởi sự cố này. Thứ hai, luôn cần phân biệt "số đo trong lúc huấn luyện" với "đánh giá lại từ tệp trọng số đã lưu"; ma trận nhầm lẫn, đường cong và ảnh dự đoán sinh ra từ các checkpoint hỏng không được sử dụng.

CHƯƠNG 8. TỔNG KẾT VÀ CÂU HỎI ÔN TẬP

8.1. Tóm tắt theo nguyên lý Pareto

Toàn bộ bài học có thể gói gọn trong một dòng chảy. Ảnh nội soi 640×640 đi qua backbone, nơi khối TSVM ở tầng 10 làm giàu đặc trưng mức P5 bằng ngữ cảnh toàn cục và thông tin hình dạng. Neck trộn ba mức P3, P4 và P5. Head Segment26 dự đoán tại mỗi vị trí một khung, một điểm tin cậy và 32 hệ số; Proto26 tạo 32 prototype; hệ số pha trộn prototype thành mặt nạ của từng polyp. Nhánh một-một giúp mỗi polyp chỉ có một dự đoán nên không cần NMS. Chất lượng được đo bằng Mask mAP@50-95, trung bình trên 10 seed và được kiểm định thống kê.

8.2. Những hiểu lầm cần tránh

- mAP@50-95 không phải là tổng AP từ ngưỡng 50% đến 95% mà là trung bình AP ở mười ngưỡng IoU cách nhau 0,05.
- Precision cao chưa chắc mô hình tốt, vì có thể mô hình báo rất ít và bỏ sót nhiều; cần xem cả Recall hoặc AP.
- TSVM không thay toàn bộ backbone bằng Mamba mà chỉ thay đúng một khối ở tầng 10.
- Không nên lập luận rằng TSVM giúp mô hình nhanh hơn khi chưa đo tốc độ thực tế.
- Nhánh "topology" là tích chập có hướng và độ lớn biên, không có ràng buộc topo tường minh.
- Hàm fuse() chỉ bỏ những nhánh không dùng khi dự đoán; sự cố seed 0, 5, 8 là do lưu thống kê chuẩn hóa batch ở định dạng FP16.
- Giá trị p bằng 0,38 không có nghĩa là có 38% khả năng TSVM tốt hơn.

8.3. Câu hỏi ôn tập kèm gợi ý trả lời

Câu 1. Với ảnh đầu vào 640×640 , ba mức đặc trưng P3, P4, P5 có kích thước bao nhiêu và mô hình đưa ra tổng cộng bao nhiêu vị trí dự đoán? Gợi ý: lần lượt là 80×80 , 40×40 và 20×20 ; tổng cộng 8.400 vị trí.

Câu 2. Mặt nạ của một polyp được tạo ra từ đâu? Gợi ý: từ 32 hệ số riêng của đối tượng nhân với 32 prototype dùng chung kích thước 160×160 , cộng lại, đưa qua sigmoid, lấy ngưỡng 0,5 và cắt theo khung.

Câu 3. Vì sao YOLO26 không cần NMS, và hàm fuse() làm gì? Gợi ý: nhánh một-một được huấn luyện để mỗi đối tượng chỉ có một dự đoán; nhánh một-nhiều chỉ hỗ trợ việc học và bị fuse() xóa đi cùng nhánh semseg khi đưa vào sử dụng.

Câu 4. Tính chọn lọc của Mamba nghĩa là gì? Gợi ý: các hệ số Δ , B, C được tính từ chính dữ liệu đầu vào, nên mô hình tự quyết định ghi nhớ mạnh hay lướt qua từng phần tử.

Câu 5. Vì sao VMamba quét bốn hướng thay vì một hướng? Gợi ý: quét một hướng thì những điểm ở đầu chuỗi không nhận được thông tin từ phần sau của ảnh, và các điểm kề nhau theo chiều dọc bị đặt xa nhau trong chuỗi.

Câu 6. Trong TSVM, vì sao dùng $F_M \odot (1 + G)$ mà không dùng $F_M \odot G$? Gợi ý: để những vùng có G gần 0 vẫn giữ được đặc trưng toàn cục; thông tin hình dạng chỉ được phép khuếch đại, không được phép xóa.

Câu 7. Tập kiểm định có 127 polyp, mô hình tìm đúng 112, bỏ sót 15 và báo nhầm 7. Hãy tính Precision và Recall. Gợi ý: Precision = $112 / 119 \approx 0,941$; Recall = $112 / 127 \approx 0,882$.

Câu 8. Vì sao Mask mAP@50-95 thấp hơn nhiều so với Mask mAP@50? Gợi ý: ở các ngưỡng IoU cao từ 0,75 đến 0,95, dự đoán phải gần như trùng khít; mô hình tìm đúng polyp nhưng viền chưa đủ sát.

Câu 9. Có được viết "TSVM cải thiện đáng kể so với mô hình gốc" không? Gợi ý: không, vì $p = 0,3839$ lớn hơn 0,05. Chỉ nên nói TSVM cao hơn một chút, ổn định hơn và chênh lệch chưa có ý nghĩa thống kê.

Câu 10. Vì sao các checkpoint TSVM seed 0, 5, 8 hỏng khi đánh giá lại, trong khi số liệu 10 seed vẫn dùng được? Gợi ý: giá trị trung bình chạy của lớp chuẩn hóa batch ở tầng 10 vượt giới hạn FP16 nên bị cắt khi lưu; còn số liệu 10 seed được ghi trong lúc huấn luyện, trước khi checkpoint bị lưu sai.